physical_object_simulator

The Complete Solution, Explained

physical_object_simulator

July 22, 2026

Contents

1 What problem does this solve? **1**
1.1 Why "sundials-only" is a big deal . . . 1

2 The layout **2**

3 The unique union: `pub struct physical_object` **2**

4 The system: `PhysicalObjectSystem` **4**

5 Integration: how the physics advances **5**
5.1 CVODE (Adams or BDF) — the general path . . . 5
5.2 ARKODE SPRKStep — the symplectic path . . . 5
5.3 The API . . . 5

6 The front end in brief **6**
6.1 The graphical scene window . . . 6

7 How we know it is right **7**

8 Fourteen more worked examples **7**
8.1 S1 — build, run, and read a report (the canonical loop) . . . 8
8.2 S2 — reading eccentricity off the Laplace vector . . . 8
8.3 S3 — retiring the legacy Euler: `integrate()` on one body . . . 9
8.4 S4 — a charge in a uniform E field (exact answer) . . . 9
8.5 S5 — static bodies: the `inverse_mass = 0` convention . . . 9
8.6 S6 — geometry: signed distances, normals, frames . . . 10
8.7 S7 — the SPRK separability gate as an API . . . 10
8.8 S8 — driving the simulator from Python (no Jupyter) . . . 10
8.9 S9 — the same physics from a JupyterLab notebook . . . 11
8.10 S10 — choosing an integrator: an honest benchmark . . . 11
8.11 S11 — watching a tumbling cuboid in the scene window . . . 12
8.12 S12 — driving the scene from JupyterLab (async events) . . . 12
8.13 S13 — the box of shapes (every variant in one rigid box) . . . 13

8.14 S14 — two dumbbells: a user-defined function builds them, and E, P and L survive the impact 14

9 Building, running, testing **15**

10 Collisions (new) **16**

11 Limitations (told straight) **17**

Written for a reader who has never seen this project, Rust, or a numerical integrator before. The companion document ***grammar.pdf*** *specifies the command language; this one explains the whole system underneath it.*

1 What problem does this solve?

The repository previously contained **three separate, overlapping ways** to describe a physical thing:

legacy type	file	what it modeled	its integrator
`PointParticle`	`src/solver.rs`	gravitating point mass; momentum, angular momentum, Laplace-vector observables	hand-written velocity Verlet
`RigidBody`	`RigidBody.rs`	full 3-D rigid body: quaternion, inertia tensors, charge, magnetic coupling, SDF boundary	hand-written semi-implicit Euler
`RigidBody3D`	`RigidBody3D.rs`	simpler rigid body: arrays, diagonal inertia from shape, sphere/cuboid enum	hand-written explicit Euler

The task: refactor into **one** pure-Rust simulator in `./physical_object_simulator`, whose single object type `physical_object` is the *unique union* of all three (every data member and property, nothing duplicated), whose *only* numerical backend is the local `./sundials_rs` library — zero `unsafe`, zero external dependencies, zero warnings — driven by a *lexer + grammar + stack machine* command language and a *notebook* interface, bridged to *JupyterLab*.

1.1 Why "sundials-only" is a big deal

SUNDIALS is Lawrence Livermore's industrial-strength suite of differential-equation solvers. `../sundials_rs` is a pure-Rust, line-faithful translation of SUNDIALS 7.8.0 — itself zero-unsafe and dependency-free — providing CVODE (adaptive multistep) and ARKODE (including symplectic integrators). The legacy Euler/Verlet steppers were *not carried over*: every trajectory now comes from a solver with proper error control or symplectic structure. The difference is not academic — explicit Euler gains energy every orbit; CVODE at default tolerances conserves the outer solar system's energy to a part in 10^6 over 1,369 years (§7).

2 The layout

```
rustSolveIt_Win11_SUNDIALS_7_8_0/
```

```
|- Cargo.toml            workspace: two member crates, no external deps
|- PLAN.md               design record
|- grammar.md/.tex/.pdf  command-language spec
|- physical_object_simulator.md/.tex/.pdf   this document
|- ARCHITECTURE.md       module architecture & contracts
|- CLAUDE.md             contributor/agent working rules
|- physical_object/      THE LIBRARY
|    src/linalg.rs           Vec3, Mat3, Quat (pure std)
|    src/boundary.rs         Boundary enum + Sdf trait + inertia
|    src/physical_object.rs  pub struct physical_object (the union)
|    src/system.rs           PhysicalObjectSystem
|    src/integrate.rs        ALL integration (CVODE + ARKODE SPRK)
|    examples/ + tests/
|- posim/                THE FRONT END (binary posim.exe)
|    src/{lexer,parser,vm,notebook,machine,main}.rs
|    src/scene/          THE GRAPHICAL SCENE WINDOW
|        mod.rs              server + playback thread + shared state
|        ws.rs               hand-rolled SHA-1 / base64 / RFC 6455
|        scene.html          the embedded window page (toolbar, canvas,
|                            status bar -- served to your browser)
|- jupyter/              JupyterLab wrapper kernel + tests
|- sundials_rs/          the pure-Rust SUNDIALS engine (own workspace)
```

Dependencies point one way: `posim` → `physical_object` → {`sundials_core`, `cvode_rs`, `arkode_rs`}. The workspace `Cargo.lock` lists **exactly five crates** — the machine-checkable proof of "zero external dependencies."

3 The unique union: `pub struct physical_object`

"Unique union" means: take every data member and property of the three legacy types; where two describe *the same physical quantity*, store it **once** and reconcile the interfaces. Twelve stored fields result:

field	from	unification decision
`id`	PointParticle	user label
`mass`	all three	coupled to `inverse_mass`
`inverse_mass`	RigidBody3D	$m \le 0 \Rightarrow 1/m := 0 =$ **static body**
`charge`	RigidBody(3D)	
`position`	all three	one `Vec3` replaces nalgebra and `[f64;3]`
`orientation`	RigidBody	always unit (setters renormalize)
`momentum`	RigidBody(3D)	**canonical**. PointParticle stored `velocity`; storing both would duplicate one quantity, so velocity became a *derived property*: reads $p\,m^{-1}$, writes $p := mv$
`angular_momentum`	RigidBody(3D)	the **spin** state; PointParticle's *orbital* method collided with this name → renamed `orbital_angular_momentum(com)`; `total_angular_momentum` = orbital + spin
`inertia_tensor`	RigidBody + 3D	one body-frame tensor

`inverse_inertia_tensor`	both	coupled; singular → zero inverse = "cannot rotate"
`magnetic_moment_tensor`	RigidBody	torque $(RMR^{\mathsf{T}})B$
`boundary`	both	RigidBody had `Box<dyn Boundary>` (SDF trait); RigidBody3D an enum. The **enum** keeps the name — **seven variants** now: the legacy `Point`/`Sphere`/`Cuboid` plus `Torus{ring_radius, tube_radius}`, `Disk{radius}`, `Cylinder{radius, half_height}` and the compound `Dumbbell{r1, r2, rod_radius, z1, z2, f1, f2}` (two solid spheres plus a solid rod as ONE rigid body; the stored mass fractions `f1`/`f2` keep every part mass recoverable), each with an exact SDF; the SDF behavior survives as the `Sdf` trait (verbatim central-difference normal) implemented *for* the enum — no trait objects, struct stays `Clone`

Every field has public `get_x()`/`set_x()` (coupled ones maintain their invariants on every write), plus derived `velocity` and `angular_velocity` ($\omega = R\,I^{-1}R^{\mathsf{T}}L$) properties.

Methods carried over verbatim: `momentum()`, `orbital_angular_momentum(com)`, `laplace_vector(com, k)` (incl. the $r = 0$ guard), `linear_velocity()`, `angular_velocity()`, `kinetic_energy()` ($= \frac{1}{2}m|v|^2+\frac{1}{2}\omega\cdot L$), `to_local_space`/`to_world_space`, `signed_distance`/`surface_normal`, `recompute_inertia_fr` (sphere $\frac{2}{5}mr^2$; cuboid $\frac{m}{3}(h^2+h^2)$ diagonals; Point → zero; the shapes added by the TORUS/DISK/-CYLINDER and DUMBBELL releases get the closed forms below).

Each of the non-legacy shapes has an exact SDF and an exact analytic inertia tensor (body frame; z is the shape's symmetry axis):

shape	**parameters**	**analytic inertia**
`Torus`	`ring_radius` c (centerline circle), `tube_radius` a	$I_z = m\,(c^2 + \frac{3}{4}a^2)$; $I_x = I_y = m\,(\frac{1}{2}c^2 + \frac{5}{8}a^2)$
`Disk`	`radius` a — ideal, zero thickness	$I_x = I_y = \frac{1}{4}ma^2$; $I_z = \frac{1}{2}ma^2$ (perpendicular-axis theorem: $I_z = I_x + I_y$)
`Cylinder`	`radius` r, `half_height` h (full height $2h$)	$I_z = \frac{1}{2}mr^2$; $I_x = I_y = m\,(3r^2 + 4h^2)/12$
`Dumbbell`	sphere radii r_1/r_2 at $(0,0,z_1)/(0,0,z_2)$, `rod_radius`, mass fractions f_1/f_2	exact **composite**: $\frac{2}{5}mr^2$ for each sphere plus its parallel-axis term mz^2, plus the rod's cylinder terms about the COM

The dumbbell — two solid spheres joined by a solid rod, **one** rigid body — is built by `boundary::dumbbell(m1, m2, m_rod, r1, r2, rod_radius, length)`, which places the sphere offsets at $z_1 = -(m_2 + m_{\text{rod}}/2)\,L/M$ and $z_2 = (m_1 + m_{\text{rod}}/2)\,L/M$ (L the center-to-center length, $M = m_1 + m_2 + m_{\text{rod}}$) so that the body-frame origin **is** the center of mass: the identity $m_1z_1 + m_2z_2 + m_{\text{rod}}(z_1+z_2)/2 = 0$ holds exactly (pinned by test). Its SDF is the exact **union** — the min of the parts' SDFs — and the stored mass fractions make every part mass recoverable from the total, which is what lets the

grammar's `d.m1/.m2/.m_rod/.r1/.r2/.rod_radius/.length` member paths read *and* write the parts (a member write rebuilds total mass, COM offsets and the inertia tensor in one step).

Every variant also answers an exact **support function** in `boundary.rs`: `support_extent(u)` $= \max_{x \in B} x \cdot u$ — the farthest reach along a direction, a genuinely **directed** quantity since the dumbbell release (the dumbbell is the first non-centrally-symmetric shape: its off-center spheres make $h(u) \neq h(-u)$; every symmetric shape's closed form is unchanged; for the torus it is the support of its convex hull: a support function cannot see the hole) — `support_point(u)`, a point achieving that extent, which returns the **centroid of the supporting set** when that set is a whole face, edge, circle or cap (so a flat-on contact puts its contact point at the face center and carries no spurious lever arm), `support_rank(u)` (the supporting set's dimension: 0 = vertex, 1 = edge/circle, 2 = face/cap), and `bounding_radius()`. The collision tiers of §10 and the anti-tunnel step cap are built on exactly these.

Three constructors mirror the legacy `new` functions: `new_point`, `new_rigid`, `new_from_shape`.

Naming quirk: the struct is deliberately `physical_object` (lower-case, per the specification; legal because crate roots allow `non_camel_case_types`) and lives in a same-named module, so the import is `use physical_object::physical_object::physical_object;` and sibling paths need a leading `::`.

4 The system: `PhysicalObjectSystem`

The legacy `GravitationalSystem`, generalized: objects, `g_constant`, `softening` (legacy const → field, 10^{-6}), uniform gravity/E/B fields, per-object external force/torque, CVODE tolerances, `method`, `time`. Observables ported verbatim: `total_mass`, `center_of_mass`, `compute_accelerations` (identical softened arithmetic), plus `total_momentum`, `total_angular_momentum`, `total_energy`, `laplace_vector(i)` ($k = G\,M_{\text{total}}$, about the center of mass).

Solver packing — **13 numbers per object**:

```
[ x y z | px py pz | qw qx qy qz | Lx Ly Lz ]   per object -> length 13N
```

`pack_state`/`unpack_state` are exact inverses (unpack renormalizes quaternions).

5 Integration: how the physics advances

`integrate.rs` is the **only** file that advances time. It solves

$$\begin{aligned}
\dot{q} &= p\,m^{-1} \\
\dot{p} &= \textstyle\sum_j G m_i m_j \frac{q_j - q_i}{(|\Delta|^2 + \varepsilon^2)^{3/2}} + m\,g + qE + q\,v \times B + F_{\text{ext}} \\
\dot{\hat{q}} &= \tfrac{1}{2}\,(0, \omega) \otimes \hat{q}, \qquad \omega = R\,I^{-1} R^{\mathsf{T}} L \\
\dot{L} &= \tau_{\text{ext}} + (R M R^{\mathsf{T}}) B
\end{aligned}$$

5.1 CVODE (Adams or BDF) — the general path

`Method::Adams` (default) and `Method::Bdf` use CVODE with Newton iteration and the dense linear solver (difference-quotient Jacobian), following the reference pattern of `cvode_rs/examples/solar_system.rs`. Contracts:

- **Callbacks are plain `fn` pointers** (no closures): parameters are cloned into a snapshot, passed as `user_data: Option<Box<dyn Any>>`, downcast inside; a failed downcast returns -1 (SUNDIALS "unrecoverable"), never panics.
- **Quaternion drift**: renormalized only at output points and only when $||\hat{q}| - 1| > 10^{-10}$; the state mutation invalidates CVODE's multistep history, so the driver then calls `CVodeReInit` (accumulating statistics first — ReInit zeroes the counters).

5.2 ARKODE SPRKStep — the symplectic path

`Method::Sprk{table, dt}` uses symplectic partitioned Runge–Kutta at fixed step (state repacked `[q(3N) | p(3N)]`, following `ark_kepler.rs`). Symplectic error stays *bounded forever* instead of drifting. The legacy velocity Verlet *is* `ARKODE_SPRK_LEAPFROG_2_2`. Separability is **gated**: any B field, magnetic tensor, external torque or spinning body yields an error naming the offending feature.

5.3 The API

```
pub fn run (system: &mut PhysicalObjectSystem, t_end: f64, nout: usize)
    -> Result<RunReport, String>;
pub fn step(system: &mut PhysicalObjectSystem, dt: f64)
    -> Result<RunReport, String>;
pub fn propagate_single(obj: &mut physical_object, force: Vec3,
    torque: Vec3, dt: f64) -> Result<(), String>;
```

`RunReport` carries per-output snapshots (time, energy, momentum, angular momentum, center of mass) plus solver statistics (`nst`, `nfe`, `nni`, `netf`). `physical_object::integrate(force, torque, dt)` — the union of the two legacy Euler methods — delegates to `propagate_single`, so even "one object, one step" goes through CVODE.

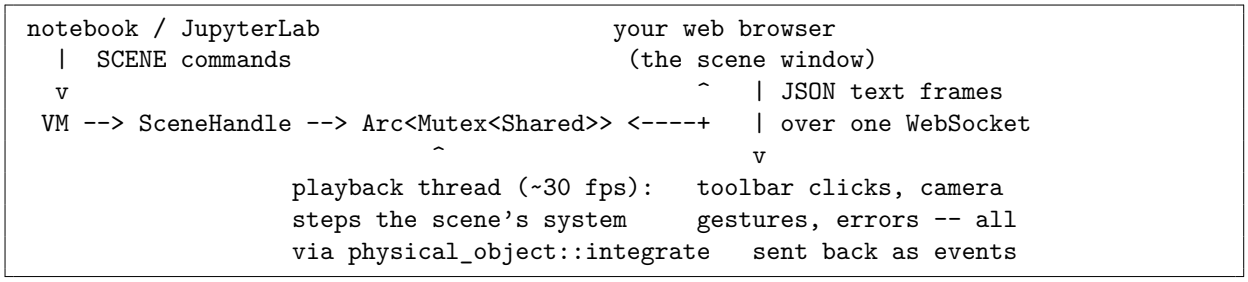
6 The front end in brief

(Fully specified in `grammar.pdf`.) `posim` runs the lexer → parser → stack-machine pipeline; the notebook REPL numbers cells `In[n]`/`Out[n]` with `%history`/`%edit`/`%rerun`/`%save`/`%load` magics; `-script` replays files; `-notebook` replays a file and then stays interactive (the dynamic-notebook launcher, scene window alive); `-machine` speaks line-delimited JSON (hand-rolled, zero dependencies). The JupyterLab bridge is a ∼100-line Python wrapper kernel subprocessing `posim -machine`.

6.1 The graphical scene window

`SCENE CREATE` opens a **separate graphical window, outside the notebook** — your web browser — showing every simulator entity. How can a zero-dependency program open a graphical window? It doesn't link a GUI library at all: posim starts a tiny **HTTP + WebSocket server** on `127.0.0.1` (written with nothing but `std::net`, including the SHA-1 and base64 the WebSocket handshake needs, in `posim/src/scene/ws.rs`) and serves one self-contained HTML/canvas page (`scene.html`, embedded into the binary at compile time). The browser supplies the pixels; posim supplies everything else. Zero `unsafe`, zero external crates, `Cargo.lock` still lists five.

The moving parts, and who talks to whom:



- **The playback thread** owns a *synchronized copy* of the notebook's system and evolves it forward at `SCENE SET_TIME_STEP`'s `dt`, ∼30 frames per second, broadcasting each frame's positions and orientations to every connected window. Time is advanced **only** through `physical_object::integrate` — the sundials-only rule holds in the scene too; there is no second stepper hiding in the GUI.
- **Reverse** (`SCENE REVERSE` or the toolbar button) replays a ring buffer of up to 20,000 snapshots recorded while stepping forward — a faithful rewind of states the solver actually produced, never "integration with negative dt".
- **Asynchronous, both directions.** Notebook → window: scene deltas, camera commands, playback state. Window → notebook: errors, data requests and user actions are queued as *events* you read with `SCENE EVENTS`; under `-machine`, each event is also pushed immediately as an unsolicited `{"event": ...}` JSON line. The Jupyter kernel runs a background reader thread that routes ordinary replies to the requesting cell and streams event lines into the notebook as `[scene] ...` output — the window can talk to you *between* cells.
- **In the window**: a toolbar (Start, Pause, Stop, Reverse, single-step, dt entry, zoom, reset view, grid/trails/labels toggles, `?` help) and a status bar (connection dot, mode, simulated time, dt, total energy, body count, hidden count, history depth, camera yaw/pitch/distance, fps). Arrow keys translate the view, left-drag rotates, the mouse wheel or `+/-` zooms; `H` shows the full cheat-sheet.

7 How we know it is right

check	result
`cargo build -workspace`	zero warnings (`#![deny(warnings)]`)
`cargo test -workspace`	52 tests green (17 library, 9 conservation, 26 posim)
`#![forbid(unsafe_code)]`	every crate root; `grep unsafe` finds nothing else
`Cargo.lock`	exactly 5 local crates ⇒ zero external deps
outer solar system	matches the donor sundials example: Pluto to **8 decimals after 500,000 days**; energy drift 7.8×10^{-7}
Kepler $e = 0.6$	energy, L, Laplace vector conserved $< 10^{-6}$ on both paths
torque-free tumbling	$\Delta L = 0$ exactly; KE drift 5×10^{-9}; $\lvert\hat{q}\rvert$ drift 0
charged gyration	gyroradius $= mv/(qB)$ to 10^{-4}; orbit closes to 7×10^{-9}
Jupyter	protocol test (24 checks incl. the SCENE family + `events` op), ZMQ kernel test (5 cells), live JupyterLab launch + cell execution
WebSocket layer	SHA-1 against the FIPS 180-4 test vectors; base64 against RFC 4648 §10; the handshake against the worked example in RFC 6455 §1.3
scene server	three integration tests drive a real TCP client: page serves with toolbar/status bar; a WebSocket session handshakes and streams frames; camera sync / start / pause / events round-trip; forward-then-reverse playback lands **exactly** back on the $t = 0$ state
scene window (real browser)	16 headless-Chrome checks dispatching genuine input events: arrow keys translate the view (and return exactly), left-drag changes yaw *and* pitch (and does not zoom), wheel and `+`/`-` zoom in-/out, toolbar Start evolves t, Pause freezes, Reverse plays t backward; the status bar reports it all

8 Fourteen more worked examples

Additional to the command-language examples in `grammar.pdf` — these exercise the **Rust API**, the **machine protocol**, **JupyterLab**, and the **graphical scene window**. Every number was produced by the real code. To run a Rust snippet, drop it into `physical_object/examples/demo.rs` (with `fn main()`) and `cargo run -p physical_object -example demo`.

Common imports:

```
use ::physical_object::boundary::Boundary;
use ::physical_object::integrate::{run, step, Method};
use ::physical_object::linalg::{Quat, Vec3};
use ::physical_object::physical_object::physical_object;
```

```
use ::physical_object::{PhysicalObjectSystem, Sdf};
```

8.1 S1 — build, run, and read a report (the canonical loop)

```
let sun   = physical_object::new_point(0, 1.0e9, Vec3::zeros(), Vec3::zeros());
let earth = physical_object::new_point(1, 1.0,
    Vec3::new(1.0, 0.0, 0.0), Vec3::new(0.0, 1.0, 0.0));

let mut sys = PhysicalObjectSystem::new(vec![sun, earth], 1.0e-9); // G*M ~= 1
sys.softening = 0.0;
sys.method = Method::Adams;

let e0 = sys.total_energy();
let report = run(&mut sys, 6.28, 4)?;          // one orbit, 4 snapshots

for s in &report.snapshots {
    println!("t␣=␣{:5.2}␣␣E-drift␣=␣{:.2e}␣␣com␣=␣{:?}",
             s.t, ((s.energy - e0) / e0).abs(), s.center_of_mass.to_array());
}
println!("solver␣steps␣=␣{},␣rhs␣evals␣=␣{}", report.nst, report.nfe);
```

What to notice. `run` takes an **absolute** end time and mutates the system in place; `RunReport` gives conserved-quantity snapshots with no extra bookkeeping. All solver errors are `Result<_, String>` — no panics in library code.

8.2 S2 — reading eccentricity off the Laplace vector

For a unit-mass orbiter with $GM = 1$, the eccentricity *vector* is $\vec{e} = \vec{A}/(mk)$: its length is the orbital eccentricity and it points at perihelion.

```
let ecc = 0.6;
let central = physical_object::new_point(0, 1.0e9, Vec3::zeros(), Vec3::zeros());
let orbiter = physical_object::new_point(1, 1.0,
    Vec3::new(1.0 - ecc, 0.0, 0.0),                        // perihelion
    Vec3::new(0.0, ((1.0 + ecc) / (1.0 - ecc)).sqrt(), 0.0));
let mut s = PhysicalObjectSystem::new(vec![central, orbiter], 1.0 / (1.0e9 + 1.0));
s.softening = 0.0;

run(&mut s, 37.0, 5)?;                                     // ~6 orbits

let k = s.g_constant * s.total_mass();                     // = 1
let m = s.objects[1].get_mass();
let e_vec = s.laplace_vector(1).unwrap() / (m * k);
// prints: e_vec = [0.600000, 0.000000, 0.000000], |e| = 0.600000
```

What to notice. After six orbits the recovered eccentricity is `0.600000` — six exact digits, using the *legacy PointParticle formula* ($A = p \times L - mk\hat{r}$, $k = GM_{\text{total}}$), now demonstrably correct because the integrator underneath no longer lies. The k-scaling makes the formula the true conserved LRL vector only for a unit-mass orbiter — hence $m = 1$ with G scaled down.

8.3 S3 — retiring the legacy Euler: `integrate()` on one body

The legacy `RigidBody3D` demo dropped a charged sphere under gravity with a "wind" torque via explicit Euler at $dt = 0.1$. The same call now runs CVODE:

```
let mut body = physical_object::new_from_shape(
    0, 2.0, -1.5,
    Vec3::new(0.0, 10.0, 0.0),        // position
    Vec3::new(1.0, 0.0, -0.5),        // velocity
    Vec3::new(0.0, 2.0, 0.0),         // angular velocity
    Boundary::Sphere { radius: 0.5 },
);
let gravity = Vec3::new(0.0, -9.81 * 2.0, 0.0);   // F = m g
let wind    = Vec3::new(0.1, 0.0, 0.0);

body.integrate(&gravity, &wind, 0.3)?;   // same signature as the legacy code
// exact: y(0.3) = 10 - 9.81*0.09/2 = 9.55855;  L = [0.03, 0.4, 0]
```

What to notice. The method signature is the legacy one, so old call sites read the same — but the result is exact to solver tolerance (the committed test asserts agreement to 10^{-8}). At $dt = 0.3$, explicit Euler's position error would be $\sim 4 \times 10^{-2}$ — six orders worse.

8.4 S4 — a charge in a uniform E field (exact answer)

Force qE on a resting charge gives $x(t) = \frac{qE}{2m}t^2$.

```
let mut ion = physical_object::new_point(0, 2.0, Vec3::zeros(), Vec3::zeros());
ion.set_charge(3.0);
let mut s = PhysicalObjectSystem::new(vec![ion], 0.0);   // gravity off (G = 0)
s.e_field = Vec3::new(0.5, 0.0, 0.0);
run(&mut s, 4.0, 1)?;
// analytic: 3*0.5/(2*2) * 16 = 6
// prints:   x = 6.000000000000, analytic = 6.000000000000
```

What to notice. Twelve matching digits: for polynomial solutions a multistep method of sufficient order is *exact* up to round-off. Note $G = 0$: every interaction (pairwise gravity, uniform fields, Lorentz, external) is independently switchable.

8.5 S5 — static bodies: the `inverse_mass = 0` convention

```
let mut anchor = physical_object::new_point(0, 5.0,
    Vec3::new(0.0, 10.0, 0.0), Vec3::zeros());
anchor.set_inverse_mass(0.0);          // static; get_mass() reports 0
let mut s = PhysicalObjectSystem::new(vec![anchor], 0.0);
s.uniform_gravity = Vec3::new(0.0, -9.81, 0.0);
run(&mut s, 3.0, 1)?;
// prints: [0.0, 10.0, 0.0]  -- three seconds of gravity, zero motion
```

What to notice. The coupled setters keep the pair consistent both ways: `set_inverse_mass(0)` back-computes $m = 0$, so both the force mg and the conversion $p\,m^{-1}$ vanish — pinned. **Caveat:** with $m = 0$ the body also stops *sourcing* pairwise gravity; use static bodies as anchors and test

charges, not as fixed suns (for a fixed sun use a huge mass ratio, as in S1/S2).

8.6 S6 — geometry: signed distances, normals, frames

```
let mut brick = physical_object::new_from_shape(0, 1.0, 0.0,
    Vec3::new(5.0, 0.0, 0.0), Vec3::zeros(), Vec3::zeros(),
    Boundary::Cuboid { half_extents: [1.0, 2.0, 3.0] });
brick.set_orientation(Quat::from_axis_angle(Vec3::new(0.0, 0.0, 1.0),
                                            std::f64::consts::FRAC_PI_2));

let world_p = Vec3::new(7.5, 0.0, 0.0);
let local_p = brick.to_local_space(&world_p);    // -> [0, -2.5, 0]
let sd      = brick.signed_distance(&local_p);   // -> 0.5
let n       = brick.surface_normal(&Vec3::new(0.0, -2.5, 0.0)); // -> [0,-1,0]
```

What to notice. The brick is rotated 90° about z, so a point "to its right" in world space is "below it" in body space: `to_local_space` performs $q^{-1}(p-x)$ and returns $[0,-2.5,0]$. The local y half-extent is 2, so the point sits 0.5 *outside*: `signed_distance` $= 0.5$ (verified). The normal comes from the legacy central-difference default ($\varepsilon = 10^{-6}$) — the RigidBody SDF machinery, alive inside an enum.

8.7 S7 — the SPRK separability gate as an API

```
let mut s = /* Kepler system as in S2 */;
s.b_field = Vec3::new(0.0, 0.0, 1.0);            // breaks separability
s.method = Method::Sprk { table: "ARKODE_SPRK_LEAPFROG_2_2".into(), dt: 1e-3 };

match run(&mut s, 10.0, 5) {
    Err(msg) if msg.contains("separable") => {
        eprintln!("SPRK␣refused:␣{msg}");
        s.method = Method::Bdf;                   // graceful fallback
        run(&mut s, 10.0, 5)?;
    }
    other => { other?; }
}
```

What to notice. The error is a *diagnosis*: "SPRK method requires a separable Hamiltonian: magnetic field B must be zero (the Lorentz force q v x B is velocity-dependent); use METHOD ADAMS or BDF." Library code can pattern-match it; notebook users read the same words. Nothing silently produces a wrong symplectic answer.

8.8 S8 — driving the simulator from Python (no Jupyter)

```
import json, subprocess
p = subprocess.Popen(["target/release/posim.exe", "--machine"],
                     stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                     text=True, encoding="utf-8", bufsize=1)   # posim speaks UTF-8;
    Windows' default is cp1252
def rpc(**req):
    p.stdin.write(json.dumps(req) + "\n"); p.stdin.flush()
    return json.loads(p.stdout.readline())
```

```
rpc(op="exec", code="new sphere { mass = 2, radius = 0.5, charge = -1.5 }")
rpc(op="set",  path="obj0.velocity", value=[1, 0, -0.5])
print(rpc(op="get", path="obj0.momentum")["result"])   # [2.0, 0.0, -1.0]
rpc(op="set",  path="system.uniform_gravity", value=[0, -9.81, 0])
rpc(op="exec", code="step 1")
state = rpc(op="state")["result"]     # whole system as one JSON document
```

What to notice. `get`/`set` are direct wire versions of the get/set API — a scripting language away from batch parameter sweeps; `state` dumps the entire system for logging or plotting from any language. This exact flow is what `jupyter/test_protocol.py` asserts (11 checks, all green).

8.9 S9 — the same physics from a JupyterLab notebook

With the kernel installed (`jupyter/README.md`), each cell is posim language; shift-enter executes. A real 3-cell session, as run through the Jupyter messaging protocol during verification:

```
Cell 1:  new sphere { mass = 2, radius = 0.5 }
         -> obj0
Cell 2:  set system.gravity = [0, -9.81, 0]
         set obj0.position = [0, 10, 0]
         step 1
         -> t = 1 (advanced by 1, 12 solver steps)
Cell 3:  get obj0.position.y
         -> 5.095000000000006
```

What to notice. Cell 2 is **multi-line**: the kernel forwards a cell one line at a time, stopping at the first error — errors arrive as red stderr text (the verification suite deliberately executes `get obj0.bogus_field` and checks that "unknown object field" comes back). Kernel restart = fresh simulator; the notebook file = your reproducible experiment.

8.10 S10 — choosing an integrator: an honest benchmark

Same Kepler orbit ($\sim$80 revolutions to $t = 500$), measured:

method	**internal steps**	$\lvert\Delta E/E\rvert$ at t=500
CVODE Adams (rtol 10^{-10})	22,194	2.5×10^{-8}
SPRK leapfrog, $dt = 0.01$	50,000	1.3×10^{-4}
SPRK McLachlan-4-4, $dt = 0.001$ (2 orbits)	12,600	9.2×10^{-14}

What to notice. Over this horizon the high-order adaptive Adams beats 2nd-order leapfrog on raw accuracy — the symplectic advantage is not "smaller error" but **bounded** error: leapfrog's 1.3×10^{-4} oscillates forever and never grows, while any non-symplectic method's error drifts secularly. For 10^6-orbit runs symplectic wins; for a thousand orbits at tight tolerance Adams is cheaper; McLachlan-4-4 is 4th-order *and* symplectic. Rules of thumb: **Adams** for smooth accuracy, **BDF** for stiffness (gyration), **SPRK** for astronomical-length point-mass runs.

8.11 S11 — watching a tumbling cuboid in the scene window

Everything here is typed into the ordinary notebook (`cargo run`); the *graphics* happen in the browser window that opens.

```
In[1]: new cuboid { mass = 2, half_extents = [0.3, 0.2, 0.1],
                    position = [0, 2.5, 0.5], angular_velocity = [3, 0.2, 0.1] }
Out[1]: obj0
In[2]: new sphere { mass = 256, radius = 0.35 }
Out[2]: obj1
In[3]: scene create
Out[3]: scene window created: http://127.0.0.1:41372/
        (opened in your browser; if no window appeared, open that
        address yourself)
        showing 2 entities; SCENE START begins the evolution
In[4]: scene set_time_step 0.002
Out[4]: scene time step dt = 0.002
In[5]: scene start
Out[5]: scene playback: running
```

What you see. A dark 3-D viewport with a ground grid and the world axes: `obj1` is a shaded sphere, `obj0` a **wireframe box** that tumbles — its long axis wobbles because ω is not aligned with a principal axis (torque-free precession, the same physics the `tumbling_body` example checks numerically). Colored **trails** show where each body has been; the status bar counts simulated time up at $dt = 0.002$ per solver step, $\sim$30 steps a second. Now steer it:

```
In[6]: scene hide 1              # sphere vanishes; the box stays
Out[6]: 1 object(s) hidden
In[7]: scene show all
Out[7]: 0 object(s) hidden
In[8]: scene rotate 45 -10       # orbit the camera: yaw +45, pitch -10
Out[8]: camera yaw = -15 deg, pitch = 45 deg
In[9]: scene zoom in
Out[9]: camera distance = 9.6
In[10]: scene pause
Out[10]: scene playback: paused
In[11]: scene reverse
Out[11]: scene playback: reversing
```

What to notice. At `In[11]` the window plays the motion **backward in time** — the box untumbles along the exact recorded states (snapshot replay, not re-integration), pausing by itself when it reaches the beginning and reporting it: `scene events` then prints `reverse: reached the beginning of recorded history -- paused`. The same controls live in the window's toolbar, and the keyboard/mouse work directly: arrow keys translate, left-drag rotates, wheel zooms.

8.12 S12 — driving the scene from JupyterLab (async events)

The scene window can talk to the notebook *between* cells — the part a synchronous REPL cannot do. With the kernel installed:

```
Cell 1:  new point { mass = 1, position = [1, 0, 0], velocity = [0, 1, 0] }
         new point { mass = 1000, position = [0, 0, 0] }
         scene create
         -> obj0
         -> obj1
         -> scene window created: http://127.0.0.1:35519/ ...
```

```
Cell 2:  scene start
         -> scene playback: running

   (you click Pause and then Stop in the browser window's toolbar;
    with no cell running, these lines appear in the notebook by
    themselves)

         [scene] window action: pause
         [scene] window action: stop

Cell 3:  scene status
         -> scene: http://127.0.0.1:35519/  (1 window(s) connected)
           mode = stopped, t = 3.762, dt = 0.01, steps = 376,
           history = 0 frame(s)
           entities = 2 (hidden: none)
           camera: yaw = -60, pitch = 55, dist = 12, target = [0, 0, 0]
```

What to notice. The `[scene]` lines are **asynchronous**: posim pushed `{"event":"scene","message":"window action: pause"}` on stdout the moment you clicked, the kernel's background reader thread caught it, and JupyterLab printed it — no cell was executing. Errors travel the same road, marked red: if the window's JavaScript throws, or you type a bad dt into its toolbar, the notebook shows e.g. `[scene] error: window sent a non-positive dt -- ignored` on stderr. In the plain terminal notebook the same events queue up silently instead (no async printing into your prompt) and `scene events` drains them on demand — one mechanism, two delivery styles.

8.13 S13 — the box of shapes (every variant in one rigid box)

The committed script `scripts/collisions/11_box_of_shapes.posim` rattles **all six shapes** inside a `BOX 4`. At the center sits a mass-1 torus (inner radius 1, outer radius 2) tilted onto the axis $(1,1,1)/\sqrt{3}$ — an *axis-aligned* torus of outer radius 2 would exactly inscribe the 4-box, but tilted, its support extent per world axis is $1.5\sqrt{2/3}+0.5 \approx 1.7247$, clearance 0.2753 (§3's `support_extent`, and a committed test). Around it: a point particle — the **only mover**, $v = (100, 200, 100)$ — a mass-2 sphere, a 2/3-mass disk, a 5/3-mass cube and a mass-2 cylinder, positions drawn by a small documented LCG inside the script. Captured session (abridged):

```
In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 --- six static walls obj0, obj1,
        obj2, obj3, obj4, obj5 with inverse_mass = 0 (infinitely
        massive); objects collide elastically off the inside faces
In[4]:= new point { mass = 1, position = [1.406590, -0.995859,
                    0.569601], velocity = [100, 200, 100] }
Out[4]= obj7
In[10]:= collide
Out[10]= collisions ON (51 collidable pair(s); 0 impulse(s) so far)
In[13]:= energy
Out[13]= 30000
In[14]:= momentum
Out[14]= [100, 200, 100]
In[15]:= run 0.1 steps 100
Out[15]= t = 0.1 (2121 solver steps, 100 snapshots, |dE/E| = 2.040e-10,
         119 collision(s) --- CONTACTS lists them)
```

```
In[16]:= energy
Out[16]= 29999.999993880127
In[17]:= momentum
Out[17]= [146.48911126803657, 102.1478121131382, 46.01601291636828]
```

What to notice. $E_0 = \frac{1}{2} \cdot 1 \cdot (100^2 + 200^2 + 100^2) =$ **30000** exactly (one mover). In 0.1 s the box produces **119 collisions** spanning all three dispatch tiers of §10, and total energy survives every one of them to $|\Delta E/E| = 2.040 \times 10^{-10}$. Momentum does **not** survive — $(100, 200, 100) \to (146.49, 102.15, 46.02)$ — and that is the *physical signature* of the infinitely massive walls: they absorb momentum without moving ($\Delta v = \Delta p\, m^{-1} = \Delta p \cdot 0$), and after the run all six slabs are still bit-identically at rest. The 51 collidable pairs are $\binom{12}{2} = 66$ minus the 15 static wall–wall pairs, which are skipped. In the scene window the same setup shows the new wireframes — torus, disk, cylinder, quaternion-rotated — plus a dashed interior box outline; the wall slabs themselves are not drawn as bodies.

8.14 S14 — two dumbbells: a user-defined function builds them, and E, P and L survive the impact

The committed script `scripts/collisions/12_two_dumbbells.posim` defines `create_dumbell(...)` **in the notebook language** (`DEF` — eleven parameters, all but the name with defaults) and calls it twice to build two **named** rigid dumbbells — two solid spheres joined by a solid rod, one rigid body whose local origin is its center of mass (§3). They approach off-center with spin and collide twice, elastically, with $G = 0$. Abridged captured session (the full transcript is Example 12 of `collision_detection.pdf`):

```
In[3]:= create_dumbell("dumbell0", 1, 2, 0.5, 0.25, 0.25, 0.1, 1,
                       [-2, 0.15, 0], [1.5, 0, 0], [0, 0, 0.6])
Out[3]= obj0 as dumbell0
In[4]:= create_dumbell("dumbell1", 2, 1, 0.4, 0.3, 0.2, 0.08, 1.2,
                       [2, -0.15, 0], [-1.5, 0, 0], [0.4, 0, 0])
Out[4]= obj1 as dumbell1
In[9]:= energy
Out[9]= 7.865310611764706
In[10]:= momentum
Out[10]= [0.15000000000000036, 0, 0]
In[11]:= angmom
Out[11]= [0.4443030588235295, 0, -1.5059999999999998]
In[12]:= run 3 steps 60
Out[12]= t = 3 (3861 solver steps, 60 snapshots, |dE/E| = 9.463e-11,
         2 collision(s) --- CONTACTS lists them)
In[13]:= energy
Out[13]= 7.865310611020375
In[14]:= momentum
Out[14]= [0.15000000000000036, 0, 0]
In[15]:= angmom
Out[15]= [0.44430305882373156, -0.00000000000000095479180117763 46,
         -1.505999999999855]
```

What to notice. Through two real CVODE collision events, energy drifts by $|\Delta E/E| = 9.463 \times 10^{-11}$ (printed by the run line itself), momentum is **bit-identical**, and total angular momentum

about the origin drifts at the 10^{-13} level — **all three conserved at once**, unlike Example S13, where the infinitely massive walls legitimately ate momentum. Nothing here is static, and each impulse pair $\pm J\hat{n}$ acts at **one shared contact point**, so the two angular impulses about any origin cancel exactly: action–reaction produces zero net torque (the anchor test `colliding_dumbbells_conserve_energy` pins E, P and L to 10^{-8}). In the scene window the entity labels show the registered user names (`dumbell0`, `dumbell1`), the dumbbells draw as two shaded spheres at their rotated COM offsets joined by the rod's four silhouette lines, and the window's permanent labeled "conserved quantities" readout displayed `E = 7.86531061, P = [0.15000, 0.00000, 0.00000] |.| = 0.15000, L = [0.44430, 0.00000, -1.50600] |.| = 1.57017` identically before and after the impact (after it, L's y component read -1.31228×10^{-13}).

9 Building, running, testing

PowerShell (Git Bash takes the same lines with forward slashes and `POSIM_NO_BROWSER=1` *command* for the environment variable):

```
cd rustSolveIt_Win11_SUNDIALS_7_8_0
cargo run                                # the notebook (type HELP)
cargo test --workspace                   # all 638 tests (incl. scene server)
cargo build --release                    # -> target\release\posim.exe
cargo run -p physical_object --release --example kepler_orbit
cargo run -p physical_object --release --example outer_solar_system
cargo run -p physical_object --release --example tumbling_body
cargo run -p physical_object --release --example charged_in_b_field
cargo run -p posim -- --script my_session.posim
cargo run -p posim --release -- --notebook dynamic_notebooks\kepler_orbit.posim
cargo run -p posim -- --machine

# the graphical scene window: inside the notebook type
#   scene create        (opens http://127.0.0.1:<port>/ in your browser via cmd /C
    start)
# $env:POSIM_NO_BROWSER = "1" suppresses the browser launch (headless runs/CI);
# the URL is always printed, so you can open it by hand.

python jupyter\test_protocol.py        # machine-protocol checks (stdlib
                                       # only, incl. SCENE + events op)

# the JupyterLab kernel (the "posim (physical_object)" tile in the launcher)
cd jupyter
uv venv .venv
uv pip install -p .venv\Scripts\python.exe ipykernel jupyterlab
.venv\Scripts\jupyter.exe kernelspec install --user kernelspec --name posim
$env:PYTHONPATH = "$PWD"
.venv\Scripts\jupyter.exe lab
cd ..
```

10 Collisions (new)

Rigid-body collision detection is **on by default in every scene**: `STEP`, `RUN` and the scene window detect impacts *during* the time step by SUNDIALS event rootfinding (the integrator lands on the

exact time of impact), resolve them with an impulse along the **contact normal** — the action–reaction line, exposed as `contactK.normal` — and continue. Per-object `restitution` (default 1 = elastic), the `COLLIDE/CONTACTS` commands, and the full science reference (seven simulators surveyed, comparison chart, porting evaluation, twelve worked examples with captured output) live in `collision_detection.pdf`. Quick taste:

```
In [1]: new sphere { mass = 1, radius = 0.5, position = [-2,0,0], velocity = [1,0,0] }
In [2]: new sphere { mass = 1, radius = 0.5, position = [2,0,0], velocity = [-1,0,0] }
In [3]: step 3
Out[3]= t = 3 (advanced by 3, 26 solver steps, 1 collision(s) --- CONTACTS lists them)
In [4]: get contact0.normal
Out[4]= [1, 0, 0]
```

The shape tiers. With the `Boundary` enum at seven variants (§3; created with `NEW TORUS|DISK|CYLINDER|DUMBB { ... }` — `grammar.pdf` has the parameter grammar), `collide.rs` dispatches every pair in three exactness tiers — still no GJK/EPA machinery anywhere:

1. **Ball vs anything — exact, via SDF closest points.** A *ball* (a `Sphere`, or a `Point` as the zero-radius case) is tested against every shape through that shape's exact signed-distance field: the separation is $\mathrm{sdf}(\mathrm{center}) - r$, and the contact normal and point come from the exact closest surface point. Because the true distance field is used, a small ball genuinely **threads a torus hole**, and a point particle passes straight through the ideal zero-thickness disk (its unsigned distance never crosses zero — see the Limitations section).
2. **Cuboid vs cuboid — exact 15-axis SAT**, unchanged from the collision release.
3. **Extended vs extended — support-axis tests.** Every remaining pair (torus/disk/cylinder against each other or against a cuboid) is tested along candidate axes: each cuboid's three face axes, each round shape's symmetry axis, their pairwise cross products, the radial rejection axes (the component of the center offset perpendicular to each primary axis — the true lateral direction when round shapes sit side by side with parallel axes), and the center line, with the gap computed from §3's exact support extents. Exact for face-on contacts — in particular **every wall-face contact** — and exact for **side-side contacts of parallel and near-parallel round shapes** (two parallel cylinders separate by exactly their lateral gap); only a genuinely **skew corner-on** approach remains conservative (contact may register slightly early, along the nearest candidate axis). The contact point is the **support point of the lower-support-rank body** — a tilted cylinder against a wall face contributes its rim point, not the wall's face centroid; when the ranks tie, the body with the clearly smaller flat footprint wins, so a small cap landing on a big wall face contacts at the cap center. At this tier the torus is its convex hull: only balls thread the hole.

Compound bodies decompose. The dumbbell never reaches a tier as a whole: the narrow phase decomposes dumbbell-vs-anything over its parts — each sphere part is a ball tested through the other shape's exact SDF (tier 1, exact against everything, *including another dumbbell*, whose union SDF is exact), the rod recurses as a free-standing cylinder, and the deepest part supplies the contact; only **rod-vs-rod** ever reaches the approximate tier 3. Dynamically the dumbbell stays **one** rigid body, and because the impulse pair acts at one shared contact point, two colliding dumbbells conserve E, P *and* L (Example S14).

The rigid box: `BOX <size>` | `BOX OFF` | `BOX`. One command walls the world in: six **static `Cuboid` wall slabs** enclosing an inner *size* × *size* × *size* cube. "Infinitely massive" is exact, not an

approximation, because the equations of motion only ever consume the **inverse** mass: velocity is read as $v = p\,m^{-1}$ (§3), and the collision impulse divides by $n{\cdot}Kn = m_i^{-1} + m_j^{-1} + (\text{angular terms})$. A wall has `inverse_mass = 0` and zero inverse inertia, so it contributes **0** to that denominator and receives no state writes — bodies bounce elastically off the inside faces while the walls stay bit-identically at rest. The measurable consequence: energy is conserved, **momentum is not** — the walls absorb it (Δp is finite but $\Delta v = \Delta p \cdot 0 = 0$), exactly as a rigid room should. Bare `BOX` reports status, `GET system.box` reads the inner size (0 = none), and `LIST` tags each slab `[wall: static, inverse_mass=0]`. The slabs are ordinary objects with ordinary `objN` handles (see the Limitations section). Example S13 puts all six shapes inside one.

11 Limitations (told straight)

- **Collision response is restitution-only** (normal impulses; §10). Coulomb friction is designed and documented in `collision_detection.pdf` but not shipped yet; contacts are single deepest-point (no persistent multi-point manifolds), so tall box stacks are not this build's target.
- **Support-axis contacts are conservative on skew corners.** Extended-vs-extended pairs (torus/disk/cylinder against each other or a cuboid) are tested along candidate axes only (§10, tier 3): exact for face-on and wall-face contacts **and** for side-side contacts of parallel/near-parallel round shapes (the radial rejection axes), but a genuinely skew corner-on approach may register contact slightly early, along the nearest candidate axis — a convex-hull-level answer. In particular that tier sees the torus's convex hull, so **only balls (spheres/points) can thread the torus hole**. The dumbbell decomposes over its parts (§10), so its sphere contacts are exact against everything; only **rod-vs-rod** — two dumbbells meeting rod against rod — remains a support-axis pair and shares the skew-corner caveat.
- **The ideal disk is transparent to point particles — and to a parallel ideal disk.** A zero-thickness disk meets a zero-radius point in a measure-zero event — the disk's unsigned distance touches 0 without crossing it, so the rootfinder sees no sign change and the point passes through. Two disks with **parallel planes** are the same case squared: both have zero extent along the shared normal, so their separation is $|dz|$ and a face-on disk-disk crossing produces no root either (pinned by the test `parallel_disk_disk_separation_is_the_documented_limitatio`). Any ball of radius $r > 0$ collides with a disk normally; give a "point" a tiny sphere radius — or tilt one disk, or model it as a thin cylinder — if you need the crossing to fire.
- **`BOX` walls are real objects.** The six slabs occupy ordinary `objN` handles (obj0–obj5 when the box is created first) and shift the indices of objects created after them; `BOX OFF` removes them and renumbers, `DEL` keeps the tracked wall indices renumbered, and `LIST` tags them `[wall: static, inverse_mass=0]` so you can always tell which is which. Deleting a wall **dissolves** the box (`system.box` reads 0) but the surviving slabs stay tracked — `LIST` keeps their `[wall]` tag, bare `BOX` reports the dissolved state, and `BOX <size>` / `BOX OFF` replaces / removes them without leaking an orphan slab.
- **A failed `NEW` leaves nothing behind.** The initializer list is transactional: if any initializer — or the final validation, e.g. a torus with inner $\geq$ outer — fails, the half-built object is removed with the error (no ghost objects). Torus geometry in the initializer is resolved once at the closing brace, which is what makes the `inner_radius`/`outer_radius` pair genuinely order-independent.

- **Magnetic torque is not potential-derived**: energy is not conserved while $M \cdot B$ acts (by design, matching legacy semantics).
- **SPRK is translational-only** — enforced by the gate.
- **Static bodies** ($1/m = 0$) stop sourcing pairwise gravity, because their stored mass is 0 — `BOX` walls included.
- **`DEL` renumbers** subsequent object handles.
- The terminal notebook has no cursor-key cell editing (pure `std`); JupyterLab provides that experience.
- **Scene reverse is bounded**: the history ring keeps the last 20,000 forward frames; you cannot rewind past what was recorded (and `SCENE STOP` clears the ring).
- **The scene evolves a *copy*** of the notebook's system. Notebook `STEP`/`RUN` do not move the window, and scene playback does not move the notebook's state — `SCENE REFRESH` re-syncs the window from the notebook whenever you want them aligned.
- **Scene numeric arguments are term-level**: `scene rotate 15 -5` means "yaw 15, pitch -5"; to pass a sum, parenthesize — `scene zoom (1 + 0.5)`.
- **Localhost only, one scene server per posim process** — the window is a private local page, not a shared web service; a second `SCENE CREATE` reports the existing URL instead of opening a second server.
- The window needs a browser with JavaScript enabled (any current Firefox/Chrome/Safari; nothing is fetched from the internet).